Embeddings

Tópicos em Fundamentos e Métodos da Computação

Prof. Dr. Luís Paulo Faina Garcia

Universidade de Brasília
Departamento de Ciência da Computação

What do words mean?

- Words are strings or indices w_i in a vocabulary list
  - That's not very satisfactory!
- Introductory logic classes:
  - The meaning of "dog" is DOG; cat is CAT
  - That seems hardly better!
- Lexical semantics
  - The linguistic study of word meaning

Lemmas and senses

mouse (N)

1. any of numerous small rodents…
2. a hand-operated device that controls a cursor...

A sense or concept is the meaning component of a word
Lemmas can be polysemous (have multiple senses)

Lemmas and senses

mouse (N) → **lemma and citation form**

senses
1. any of numerous small rodents…
2. a hand-operated device that controls a cursor...

A sense or concept is the meaning component of a word
Lemmas can be polysemous (have multiple senses)

Synonymy

Synonyms have the same meaning in some or all contexts.

- couch / sofa
- big / large
- automobile / car
- vomit / throw up
- water / H_20

Synonymy

Synonyms have the same meaning in some or all contexts.

- couch / sofa
- big / large
- automobile / car
- vomit / throw up
- water / H_20

Two words are synonymous if they are substitutable for one another in any sentence without changing the truth conditions of the sentence.

Synonymy

Note that there are probably no examples of perfect synonymy.

- Even if many aspects of meaning are identical
- Still may differ based on politeness, slang, register, genre, etc.

Principle of Contrast

Difference in form → Difference in meaning

Synonymy?

- water / H_20

  H_20 in a surfing guide?

- big / large
  - my big sister != my large sister

In practice, the word synonym is therefore used to describe a relationship of approximate or rough synonymy.

Similarity

When we change from **synonymy** to **similarity**, we shift from **talking about relations** between word senses (like synonymy) to **relations between words** (like similarity).

Knowing how similar two words are can help in computing how similar the meaning of two phrases or sentences are.

- car, bicycle
- cow, horse

Ask humans how similar 2 words are

word1	word2	similarity
vanish	disappear	9.8
behave	obey	7.3
belief	impression	5.95
muscle	bone	3.65
modest	flexible	0.98
hole	agreement	0.3

SimLex-999 dataset (Hill et al. 2015)

Word Relatedness

Also called **word association**.

The meaning of two words can be related in ways other than similarity.

- coffee, tea: similar
- coffee, cup: related, not similar

Semantic field

A **semantic field** is a set of words which cover **a particular semantic domain** and **bear structured relations** with each other.

- field of hospitals: surgeon, nurse, hospital, etc.
- restaurants:menu, plate, food, chef, etc.

Antonymy

Senses that are **opposites** with respect to only one feature of meaning. Otherwise, they are very similar!

- dark/light
- short/long fast/slow
- rise/fall
- hot/cold
- up/down
- in/out

Connotation

Words carry emotions and judgments, in addition to their literal meaning.

Words have affective meanings

- Positive connotations (happy)
- Negative connotations (sad)

Connotations can be subtle:

- Positive connotation: copy, replica, reproduction
- Negative connotation: fake, knockoff, forgery

Connotation

Words carry emotions and judgments, in addition to their literal meaning.

Evaluation

- Positive evaluation (great, love)
- Negative evaluation (terrible, hate)

Connotation

Words seem to vary along 3 affective dimensions:

- **Valence:** the pleasantness of the stimulus
- **Arousal:** the intensity of emotion provoked by the stimulus
- **dominance:** the degree of control exerted by the stimulus

	Valence	Arousal	Dominance
courageous	8.0	5.5	7.4
music	7.7	5.6	6.5
heartbreak	2.5	5.7	3.6
cub	6.7	4.0	4.2

Osgood et al. 1957

Osgood et al. 1957

They noticed that in using these 3 numbers to represent the meaning of a word, the model was representing each word as a point in a three dimensional space.

This revolutionary idea that word meaning could be represented as a point in space.

So far

Concepts or word senses:

- Have a complex many-to-many association with words

Have relations with each other:

- Synonymy
- Antonymy
- Similarity
- Relatedness
- Connotation

Computational models of word meaning

Can we build a theory of how to represent word meaning, that accounts for at least some of the desired criteria?

We'll introduce vector semantics:

- The standard model in language processing!
- Handles many of our goals!

Vector Semantics

Vector semantics is the standard way to represent word meaning in NLP. Based on two big ideas:

1. Define the meaning of a word by its distribution in language use, meaning its neighboring words or grammatical environments.
2. Use a point in a dimensional space to represent the connotation of a word.

What ongchoi mean?

Suppose you see these sentences:

- Ongchoi is delicious sautéed with garlic.
- Ongchoi is superb over rice
- Ongchoi leaves with salty sauces

And you've also seen these:

- …spinach sautéed with garlic over rice
- Chard stems and leaves are delicious
- Collard greens and other salty leafy greens

What ongchoi mean?

Conclusion:

- Ongchoi is a leafy green like spinach, chard, or collard greens
- We could conclude this based on words like "leaves" and "delicious" and "sauteed"

Ongchoi: Ipomoea aquatica (water spinach)

Idea 1: Defining meaning by linguistic distribution

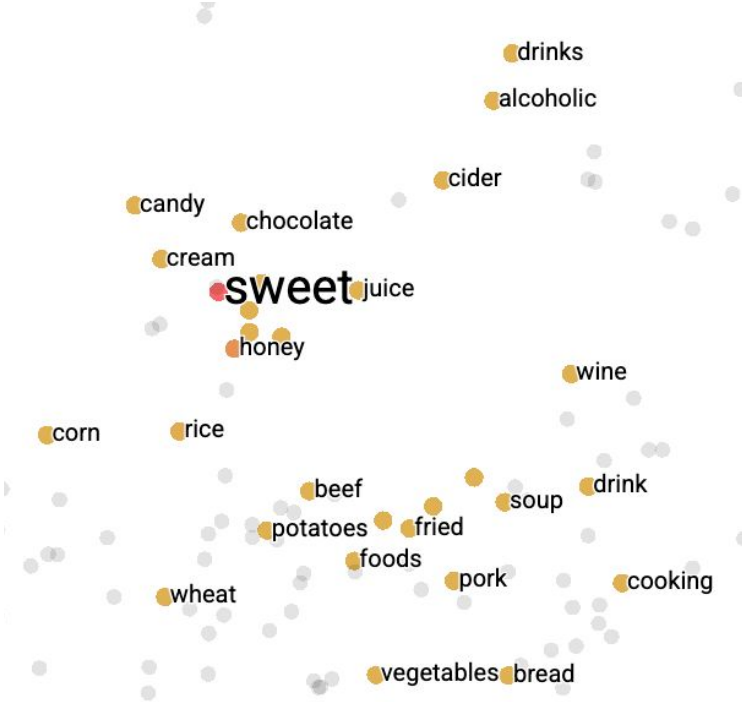
Define the meaning of a word by its distribution in language use, meaning its neighboring words or grammatical environments.

- Similar words are nearby in semantic space
- Build a space by seeing which words are nearby in text

Idea 2: Meaning as a point in space

- We define meaning of a word as a vector
- Called an embedding because it's embedded into a space
- The standard way to represent meaning in NLP
- Every modern NLP algorithm uses embeddings as the representation of word meaning

Visualization of word2vec embeddings



Two kinds of embeddings

Count-based embeddings

- Sparse vectors
- Words are represented by the counts of nearby words

Word2vec embeddings

- Dense vectors
- Representation is created by training a classifier to predict whether a word is likely to appear nearby

Count-based embeddings

Remember the intuition: words with similar neighborhoods have similar meanings

Word-context matrix (a kind of co-occurrence matrix)

- Each row represents a word in the vocabulary
- Each column represents how often each other word in the vocabulary appears nearby
- Dimensionality $|V| \times |V|$

Count-based embeddings

…is traditionally followed by **cherry** pie, a traditional dessert…

…often mixed, such as **strawberry** rhubarb pie. Apple pie…

…computer peripherals and personal **digital** assistants. These devices usually…

…a computer. This includes **information** available on the internet…

Word-Context Matrix

Co-occurrence vectors for four words with counts from the 4 windows above, showing just 3 of the potential context word dimensions.

	a	computer	pie
cherry	1	0	1
strawberry	0	0	2
digital	0	1	0
information	1	1	0

Dimensionality |V| × |V|

Word-Context Matrix

	a	computer	pie
cherry	1	0	1
strawberry	0	0	2
digital	0	1	0
information	1	1	0

- This 4x3 matrix is a subset of full |V| x |V| matrix.
- Each word is represented by a row vector with dimensionality 1 x |v|

A larger Word-Context Matrix

The dimensionality of the vector, is generally the size of the vocabulary, often between 10,000 and 50,000 words. Since most of these numbers are zero these are sparse vector representations.

	aardvark	...	computer	data	result	pie	sugar	...
cherry	0	...	2	8	9	442	25	...
strawberry	0	...	0	0	1	60	19	...
digital	0	...	1670	1683	85	5	4	...
information	0	...	3325	3982	378	5	13	...

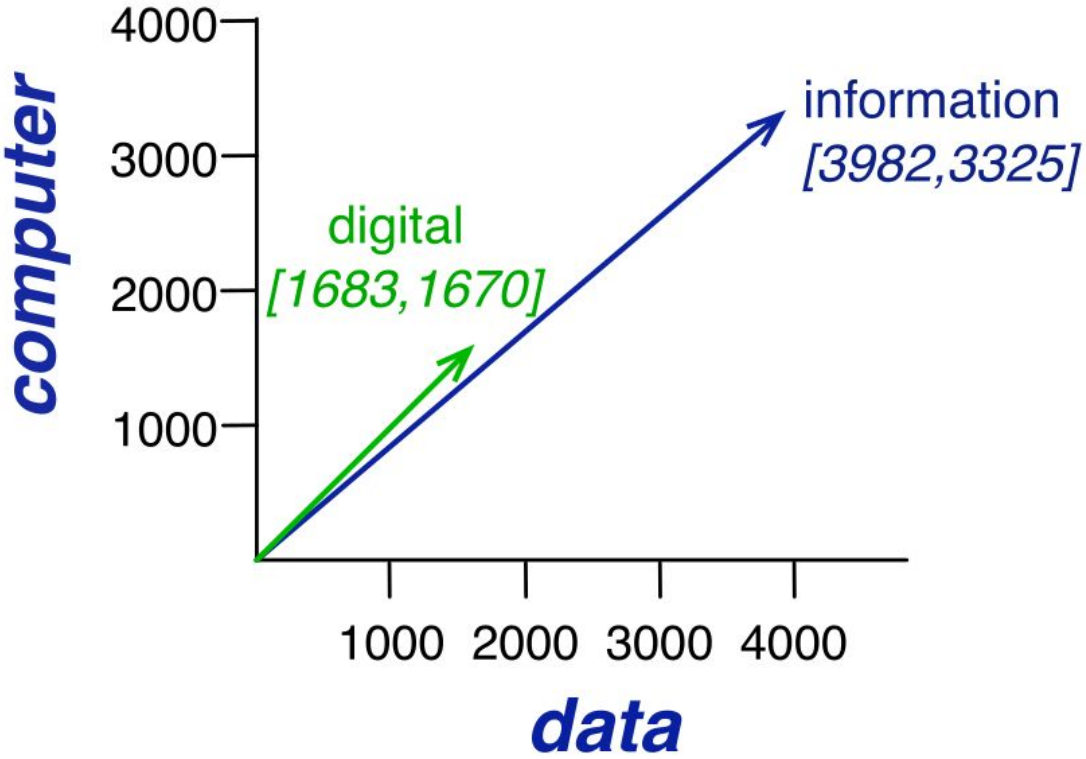
A larger Word-Context Matrix

The dimensionality of the vector, is generally the size of the vocabulary, often between 10,000 and 50,000 words. Since most of these numbers are zero these are sparse vector representations.

	aardvark	...	computer	data	result	pie	sugar	...
cherry	0	...	2	8	9	442	25	...
strawberry	0	...	0	0	1	60	19	...
digital	0	...	1670	1683	85	5	4	...
information	0	...	3325	3982	378	5	13	...

It's also possible to apply various kinds of weighting functions to the counts in these cells.

A larger Word-Context Matrix



A larger Word-Context Matrix

- Word context matrix is |V| x |V|
- This could be 50000 x 50000. Most of these numbers are zero! So these are sparse vectors.
- There are efficient algorithms for storing and computing with sparse matrices.

A larger Word-Context Matrix

- Word context matrix is |V| x |V|.
- This could be 50000 x 50000 or bigger.
- Most of these numbers are zero! So these are sparse vectors.
- There are efficient algorithms for storing with sparse matrices.
  - Linked matrix or adjacency list is more appropriate

Computing word similarity: Dot product and cosine

The dot product between two vectors is a scalar:

$$\text{dot product}(\mathbf{v},\mathbf{w}) = \mathbf{v}\cdot\mathbf{w} = \sum_{i=1}^{N} v_i w_i = v_1 w_1 + v_2 w_2 + \ldots + v_N w_N$$

The dot product tends to be high when the two vectors have large values in the same dimensions.

Dot product can thus be a useful similarity metric between vectors.

Problem with raw dot-product

Dot product favors long vectors.

Dot product is higher if a vector is longer.

Vector length:

$$|\mathbf{v}| = \sqrt{\sum_{i=1}^{N} v_i^2}$$

Frequent words (of, the, you) have long vectors (since they occur many times with other words).

So dot product overly favors frequent words

Alternative: cosine for computing word similarity

Based on the definition of the dot product between two vectors:

$$\mathbf{a} \cdot \mathbf{b} = |\mathbf{a}||\mathbf{b}| \cos\theta \quad \Longrightarrow \quad \frac{\mathbf{a} \cdot \mathbf{b}}{|\mathbf{a}||\mathbf{b}|} = \cos\theta$$

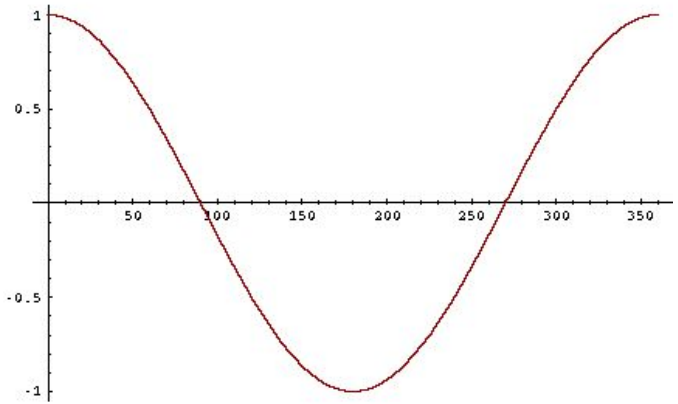
The cosine similarity metric can be computed as:

$$\text{cosine}(\mathbf{v}, \mathbf{w}) = \frac{\mathbf{v} \cdot \mathbf{w}}{|\mathbf{v}||\mathbf{w}|} = \frac{\sum_{i=1}^{N} v_i w_i}{\sqrt{\sum_{i=1}^{N} v_i^2} \sqrt{\sum_{i=1}^{N} w_i^2}}$$

Cosine as a similarity metric

- -1: vectors point in opposite directions
- +1: vectors point in same directions
- 0: vectors are orthogonal

But since raw frequency values are non-negative, the cosine for term-term matrix vectors ranges from 0-1



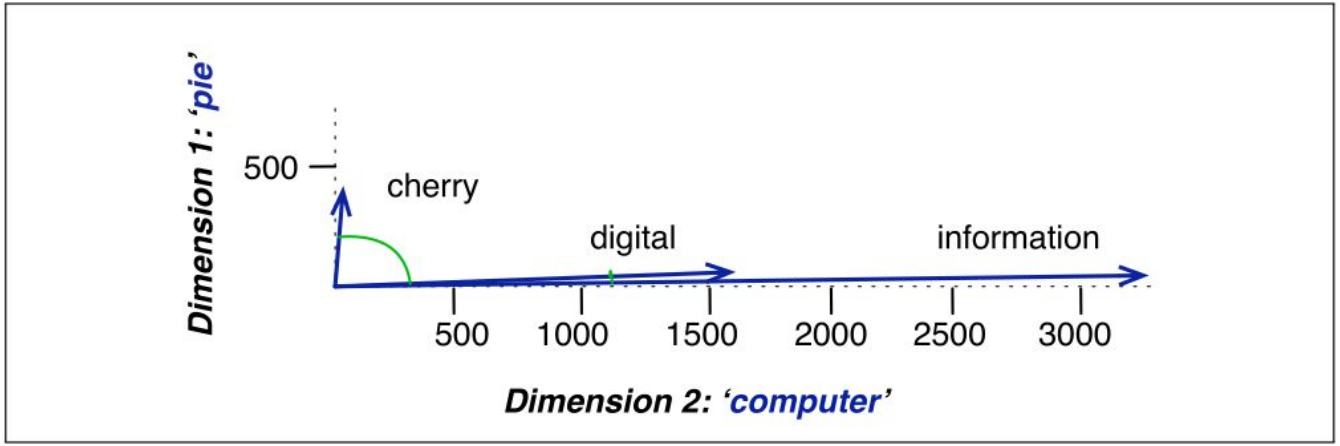
Cosine examples

	pie	data	computer
cherry	442	8	2
digital	5	1683	1670
information	5	3982	3325

$$\cos(\text{cherry}, \text{information}) = \frac{442 * 5 + 8 * 3982 + 2 * 3325}{\sqrt{442^2 + 8^2 + 2^2}\sqrt{5^2 + 3982^2 + 3325^2}} = .018$$

$$\cos(\text{digital}, \text{information}) = \frac{5 * 5 + 1683 * 3982 + 1670 * 3325}{\sqrt{5^2 + 1683^2 + 1670^2}\sqrt{5^2 + 3982^2 + 3325^2}} = .996$$

Visualizing cosines (well, angles)



Sparse versus dense vectors

Count-based embeddings

- Long (length |V|= 20000 to 50000)
- Sparse (most elements are zero)

Word2vec embeddings

- Learn vectors which are short (length 50 to 1000)
- Dense (most elements are non-zero)

Sparse versus dense vectors

Why dense vectors?

- Short vectors may be easier to use as features in machine learning (fewer weights to tune)
- Dense vectors may generalize better than explicit counts
- Dense vectors may do better at capturing synonymy:
  - car and automobile are synonyms; but are distinct dimensions
  - a word with car as a neighbor and a word with automobile as a neighbor should be similar, but aren't
- In practice, they work better!

Word2vec

- Popular embedding method
  - Static embedding
  - Very fast to train
  - Code available on the web
- Idea: predict rather than count
- Word2vec provides various options:
  - Here, we will see Skip-Gram with Negative Sampling (SGNS)

Word2vec

Instead of counting how often each word *c* occurs near "apricot", train a classifier on a binary prediction task:

- Is *c* likely to show up near "apricot"?

We don’t actually care about this task, but we'll take the learned classifier weights as the word embeddings

The big idea is self-supervision:

- A word *c* that occurs near apricot in the corpus is the correct answer for supervised learning. No need for human labels!

Word2vec Approach

1. Treat the target word *t* and a neighboring context word *c* as positive examples.
2. Randomly sample other words to get negative examples
3. Use logistic regression to train a binary classifier
4. Use the learned weights as the embeddings

Skip-Gram Training Data

Assume a +/- 2 word window, given training sentence:

...lemon, a [tablespoon of apricot jam, a] pinch...

c1 c2 [target] c3 c4

Skip-Gram Training Data

Assume a +/- 2 word window, given training sentence:

...lemon, a [tablespoon of apricot jam, a] pinch...

c1 c2 [target] c3 c4

Goal: train a classifier that is given a candidate (word, context) pair, e.g. (apricot, jam) and (apricot, aardvark), assigns each pair a probability:

- $P(+|w, c)$
- $P(-|w, c) = 1 - P(+|w, c)$

Skip-Gram Training Data

Assume a +/- 2 word window, given training sentence:

...lemon, a [tablespoon of apricot jam, a] pinch...

c1 c2 [target] c3 c4

Goal: train a classifier that is given a candidate (word, context) pair, e.g. (apricot, jam) and (apricot, aardvark), assigns each pair a probability:

- $P(+|w, c)$
- $P(-|w, c) = 1 - P(+|w, c)$

How does the classifier compute the probability P?

Similarity is computed from dot product

Two vectors are similar if they have a high dot product

- Cosine is just a normalized dot product

So:

$$Similarity(w,c) \approx \mathbf{c} \cdot \mathbf{w}$$

Turning dot products into probabilities

To turn this into a probability, we will use the sigmoid from logistic regression:

$$\sigma(x) = \frac{1}{1+\exp(-x)}$$

We model the probability that word *c* is a real context word for target word *w* as:

$$P(+|w,c) = \sigma(\mathbf{c}\cdot\mathbf{w}) = \frac{1}{1+\exp(-\mathbf{c}\cdot\mathbf{w})}$$

Turning dot products into probabilities

To turn this into a probability, we will use the sigmoid from logistic regression:

$$\sigma(x) = \frac{1}{1 + \exp(-x)}$$

We thus estimate the probability that word *c* is not a real context word for *w* as:

$$\begin{aligned} P(-|w,c) &= 1 - P(+|w,c) \\ &= \sigma(-\mathbf{c} \cdot \mathbf{w}) = \frac{1}{1 + \exp(\mathbf{c} \cdot \mathbf{w})} \end{aligned}$$

Turning dot products into probabilities

Skip-gram makes the simplifying assumption that all context words are independent, allowing us to just multiply their probabilities:

$$P(+|w, c_{1:L}) = \prod_{i=1}^{L} \sigma(\mathbf{c_i} \cdot \mathbf{w})$$

$$\log P(+|w, c_{1:L}) = \sum_{i=1}^{L} \log \sigma(\mathbf{c_i} \cdot \mathbf{w})$$

Skip-gram classifier: summary

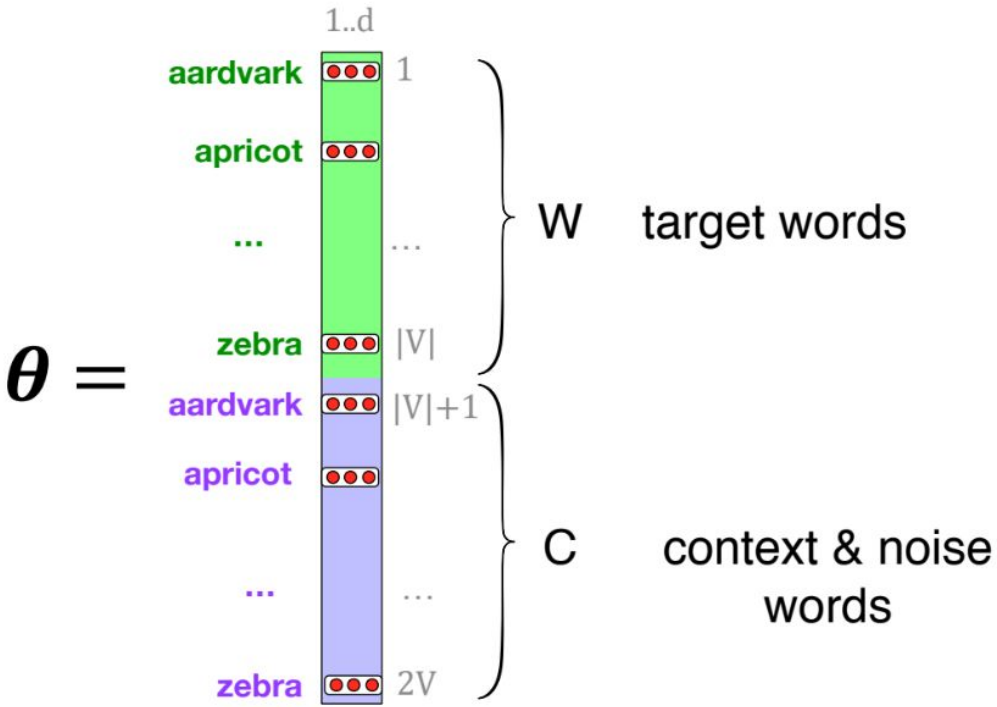
A probabilistic classifier, given

- a test target word *w*
- its context window of *L* words $c_{1:L}$

Estimates probability that *w* occurs in this window based on similarity of *w* (embeddings) to $c_{1:L}$ (embeddings).

To compute this, we just need embeddings for all the words.

Embeddings: a set for *w* and *c*



Skip-Gram Training data

...lemon, a [tablespoon of apricot jam, a] pinch...

c1 c2 [target] c3 c4

positive examples +

t	c
apricot	tablespoon
apricot	of
apricot	jam
apricot	a

Skip-Gram Training data

...lemon, a [tablespoon of apricot jam, a] pinch...

c1 c2 [target] c3 c4

positive examples +

t	c
apricot	tablespoon
apricot	of
apricot	jam
apricot	a

For each positive example we'll grab k negative examples, sampling by frequency.

Skip-Gram Training data

...lemon, a [tablespoon of apricot jam, a] pinch...

c1 c2 [target] c3 c4

positive examples +

t	c
apricot	tablespoon
apricot	of
apricot	jam
apricot	a

negative examples -

t	c	t	c
apricot	aardvark	apricot	seven
apricot	my	apricot	forever
apricot	where	apricot	dear
apricot	coaxial	apricot	if

Skip-Gram Training data

Given the set of positive and negative training instances, and an initial set of embedding vectors, the goal is to adjust those word vectors such that we:

- Maximize the similarity of the target word, context word pairs (w,c_{pos}) drawn from the positive data
- Minimize the similarity of the (w,c_{neg}) pairs drawn from the negative data.

Loss function

Two goals to be minimized: a high probability of being a neighbor for c_{pos} and high probability of being a non-neighbor for c_{neg}

$$
\begin{aligned}
L(w, c_{pos}, c_{neg*}) &= -\log\left[P(+|w, c_{pos}) \prod_{i=1}^{k} P(-|w, c_{neg_i})\right] \\
&= -\left[\log P(+|w, c_{pos}) + \sum_{i=1}^{k} \log P(-|w, c_{neg_i})\right] \\
&= -\left[\log P(+|w, c_{pos}) + \sum_{i=1}^{k} \log\left(1 - P(+|w, c_{neg_i})\right)\right] \\
&= -\left[\log \sigma(c_{pos} \cdot w) + \sum_{i=1}^{k} \log \sigma(-c_{neg_i} \cdot w)\right]
\end{aligned}
$$

Learning the classifier

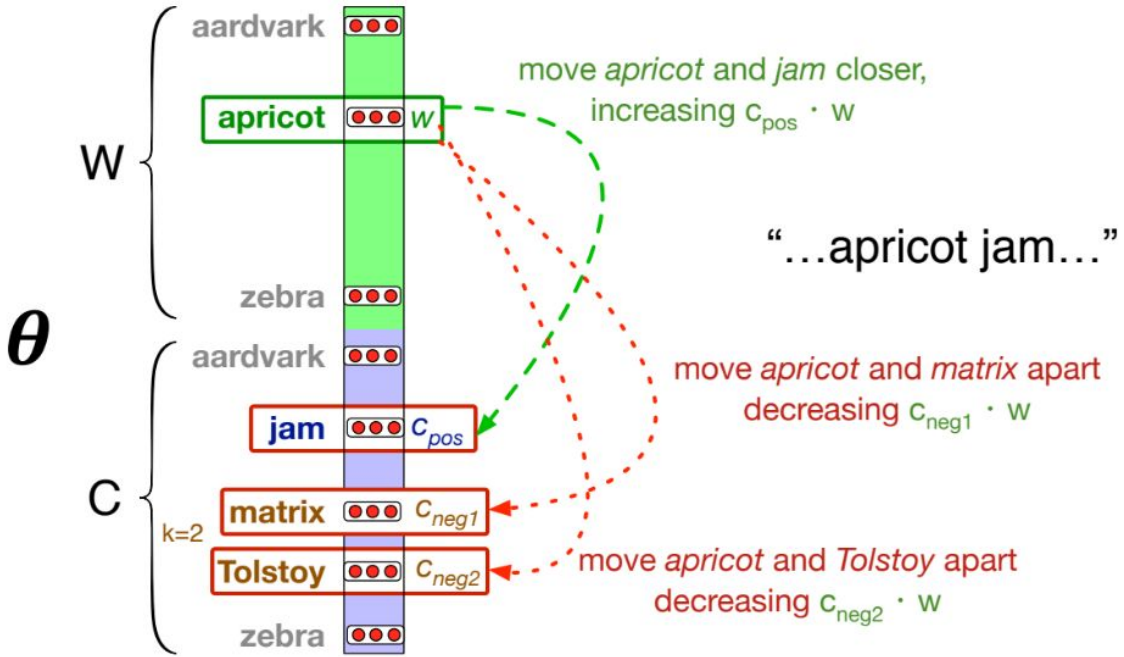
How to learn?

- Stochastic gradient descent!

We'll adjust the word weights to

- make the positive pairs more likely
- and the negative pairs less likely,
- over the entire training set.

Intuition of one step of gradient descent



Gradient descent

- At each step:
  - Direction: Move in the reverse direction from the gradient of the loss
  - Magnitude: move the value of the gradient $\frac{d}{dw}L(f(x;w),y)$ weighted by a larning rate η
  - Higher learning rate means move *w* faster

$$w^{t+1} = w^t - \eta \frac{d}{dw} L(f(x;w),y)$$

The derivatives of the loss function

$$L_{CE} = -\left[\log\sigma(c_{pos}\cdot w) + \sum_{i=1}^{k}\log\sigma(-c_{neg_i}\cdot w)\right]$$

$$\frac{\partial L_{CE}}{\partial c_{pos}} = [\sigma(c_{pos}\cdot w) - 1]w$$

$$\frac{\partial L_{CE}}{\partial c_{neg}} = [\sigma(c_{neg}\cdot w)]w$$

$$\frac{\partial L_{CE}}{\partial w} = [\sigma(c_{pos}\cdot w) - 1]c_{pos} + \sum_{i=1}^{k}[\sigma(c_{\mathrm{neg}_i}\cdot w)]c_{neg_i}$$

Update equation in SGD

Start with randomly initialized C and W matrices, then incrementally do updates

$$
\begin{aligned}
c_{pos}^{t+1} &= c_{pos}^{t} - \eta[\sigma(c_{pos}^{t} \cdot w^{t}) - 1]w^{t} \\
c_{neg}^{t+1} &= c_{neg}^{t} - \eta[\sigma(c_{neg}^{t} \cdot w^{t})]w^{t} \\
w^{t+1} &= w^{t} - \eta\left[[\sigma(c_{pos} \cdot w^{t}) - 1]c_{pos} + \sum_{i=1}^{k}[\sigma(c_{\mathrm{neg}_i} \cdot w^{t})]c_{neg_i}\right]
\end{aligned}
$$

Two sets of embeddings

SGNS learns two sets of embeddings

- Target embeddings matrix W
- Context embedding matrix C

It's common to just add them together, representing word $_i$ as the vector $w_i + c_i$

The kinds of neighbors depend on window size

Small windows (C=2): nearest words are syntactically similar words in same taxonomy

- Hogwarts nearest neighbors are other fictional schools
- Sunnydale, Evernight, Blandings

Large windows (C=5): nearest words are related words in same semantic field

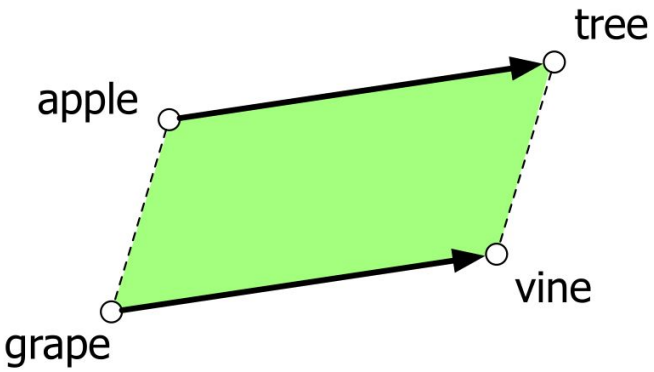
- Hogwarts nearest neighbors are Harry Potter world:
- Dumbledore, half-blood, Malfoy

Analogical relations

Another semantic property of embeddings is their ability to capture relational meanings.

To solve: "apple is to tree as grape is to _____"

Add tree – apple to grape to get vine



Analogical relations via parallelogram

The parallelogram method can solve analogies with both sparse and dense embeddings

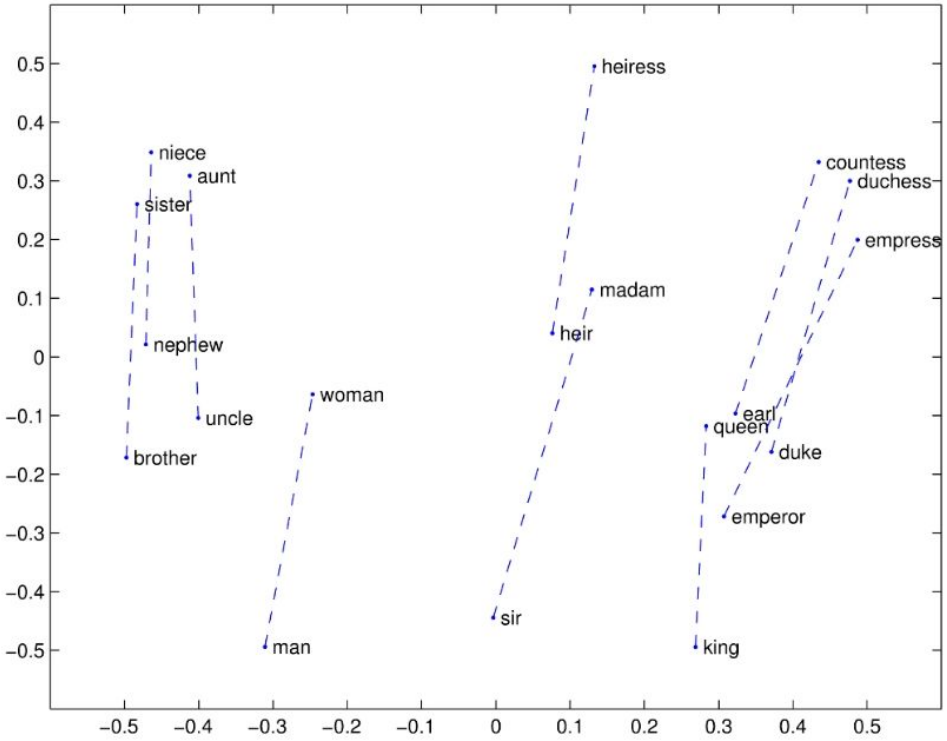
king - *man* + *woman* ~ *queen*

Paris - France + Italy ~ Rome

For a problem a:a*::b:b*, the parallelogram method is:

$$\hat{\mathbf{b}}^* = \underset{\mathbf{x}}{\operatorname{argmin}}\ \operatorname{distance}(\mathbf{x}, \mathbf{b} - \mathbf{a} + \mathbf{a}^*)$$

Analogical relations



Embeddings as a window onto historical semantics

Train embeddings on different decades of historical text to see meanings shift

Embeddings reflect cultural bias!

Ask “Paris : France :: Tokyo : x”

x = Japan

Ask “father : doctor :: mother : x”

x = nurse

Ask “man : computer programmer :: woman : x”

x = homemaker

Conclusion

- Word meaning involves multiple semantic relations.
- Vector Semantics represents meaning in a vector space.
- Models like Word2Vec learn meaning from usage.
- Dense embeddings power modern NLP.

Conclusion

Key Takeaway:

- Meaning comes from usage and can be represented as vectors.

References

- JURAFSKY, Daniel; MARTIN, James H. Speech and language processing. 3. ed. Draft. 2026. Disponível em: https://web.stanford.edu/~jurafsky/slp3/